UNIFAAT 2026.2

11 - Object Calisthenics

Prof. Luan Lourenço

Calistenia: treinar sem atalho

```
# no treino
só o peso do corpo, sem máquina

# o que se ganha
controle e técnica
```

```
# no código
sem as facilidades que escondem o problema

# o que se ganha
código que outra pessoa consegue ler e mudar
```

Restrição que ensina: **uma regra por vez**, e depois você relaxa a regra por escolha, não por ignorância.

Object Calisthenics: fail first e restrição

```
# origem
Jeff Bay, The ThoughtWorks Anthology (2008)
9 regras de restrição para treinar OO

# a pergunta de toda regra
"e se eu não pudesse fazer assim?"
```

```
# fail first
tratar o erro primeiro e sair cedo
o caminho feliz fica no fim, no nível 1

# o contrário
o código hadouken: if dentro de if dentro de if
```

- → É um **exercício**, como a escala do músico, e não uma lei da indústria: faça um projeto pequeno obedecendo tudo e veja o desenho mudar
- → Cada regra empurra uma decisão para o **dono do dado**: menos acoplamento, mais coesão
- → Nesta aula: **7 pontos**, sempre com o **jeito ruim** e o **jeito bom** lado a lado, em animação

---

Nas animações a seguir: **P** pausa, **,** e **.** andam etapa por etapa. Elas começam sozinhas.

Os 7 pontos da aula

#	ponto	jeito ruim	jeito bom
1	**Um nível de indentação** *One level of indentation*	hadouken: `if` dentro de `if` dentro de `for`	**fail first** e métodos com nome
2	**Sem** `else` *Don't use ELSE*	escada de `if`, `elif` e `else`	retorno antecipado, valor padrão, polimorfismo
3	**Obsessão por tipos primitivos** *Primitive Obsession*	`int` e `float` soltos	`Nota` e `Ra` com a regra dentro
4	**Coleção de primeira classe** *First-class collections*	lista solta circulando	`Turma`, dona da lista
5	**Um ponto por linha** (**Lei de Demeter**) *One dot per line · Law of Demeter*	`tf.pai.pai.pai.nome`	`tf.disco()`: não fale com estranhos
6	**Sem getters e setters** *Tell, Don't Ask*	pergunta o saldo e decide fora	diga: `conta.sacar(30)`
7	**Entidades pequenas** *Keep all entities small*	classe com 5 variáveis de instância	classes pequenas, até 2 variáveis

---

Ficam de fora as regras de **não abreviar** e de **tamanho de pacote**. Os códigos estão em `src/11/`.

Ponto 1: código hadouken e um nível só

One level of indentation per method · arrow code, fail first

JEITO RUIM src/11/01_indentacao.py — níveis: 4

```
1 def certificado(existe, matriculado, freq, nota):
2     if existe:
3         if matriculado:
4             if freq >= 75:
5                 if nota >= 6:
6                     return "emitido"
7     return "negado"
```

na cabeça do leitor:

existe | matriculado | freq ≥ 75 | nota ≥ 6

JEITO BOM src/11/01_indentacao.py — níveis: 1

```
1  def certificado(existe, matriculado, freq, nota):
2      if not existe:
3          return "negado: sem aluno"
4      if not matriculado:
5          return "negado: sem matrícula"
6      if freq < 75:
7          return "negado: frequência"
8      if nota < 6:
9          return "negado: nota"
10     return "emitido"
```

na cabeça do leitor:

nada pendente

> Mesma lógica: nível 4 contra nível 1, e cada saída diz o motivo.

Ainda o ponto 1: extrair o aninhado em métodos com nome

One level of indentation per method · extract method

JEITO RUIM src/11/01_indentacao.py — níveis: 3

```python
def aprovados(turma):
    aprovados = []
    for nome in turma:
        if len(turma[nome]) > 0:
            soma = 0
            for nota in turma[nome]:
                soma += nota
            if soma / len(turma[nome]) >= 6:
                aprovados.append(nome)
    return aprovados
```

na cabeça do leitor:

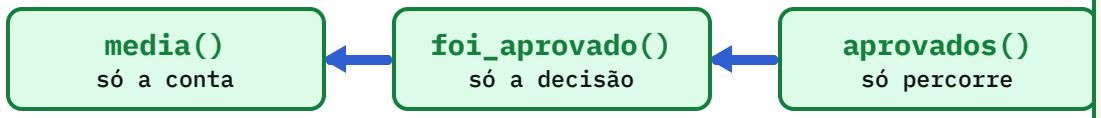
`for nome` `tem notas?` `for nota` `média ≥ 6?`

JEITO BOM src/11/01_indentacao.py — níveis: 1

```python
def media(notas):
    return sum(notas) / len(notas)

def foi_aprovado(notas):
    if len(notas) == 0:
        return False
    return media(notas) >= 6

def aprovados(turma):
    return [nome for nome in turma
            if foi_aprovado(turma[nome])]
```



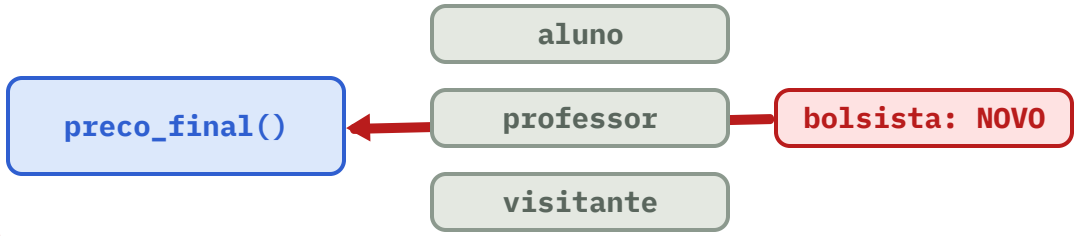
> Mesma lógica. Agora media() pode ser testada sozinha e o nome explica cada teste.

Ponto 2: sem else

Don't use the ELSE keyword

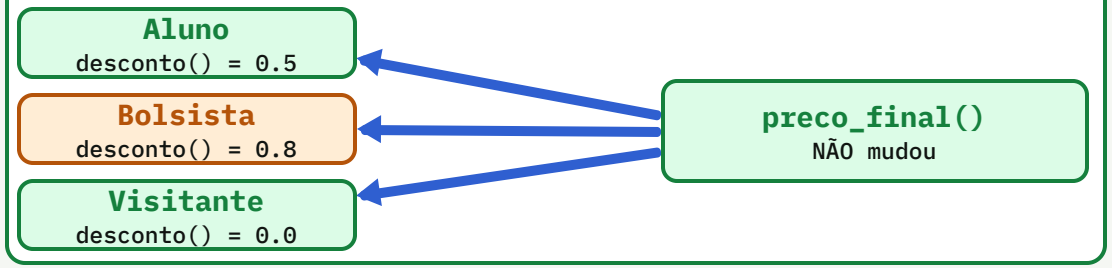
JEITO RUIM src/11/02_sem_else.py — mexe em código velho

```python
def preco_final(preco, tipo):
    if tipo == "aluno":
        d = 0.5
    elif tipo == "professor":
        d = 0.3
    elif tipo == "visitante":
        d = 0.0
    elif tipo == "bolsista":
        d = 0.8
    return preco * (1 - d)
```



JEITO BOM src/11/02_sem_else.py — só acrescenta

```python
class Aluno:
    def desconto(self): return 0.5
class Bolsista:
    def desconto(self): return 0.8
class Visitante:
    def desconto(self): return 0.0
def preco_final(preco, cliente):
    return preco * (1 - cliente.desconto())
```



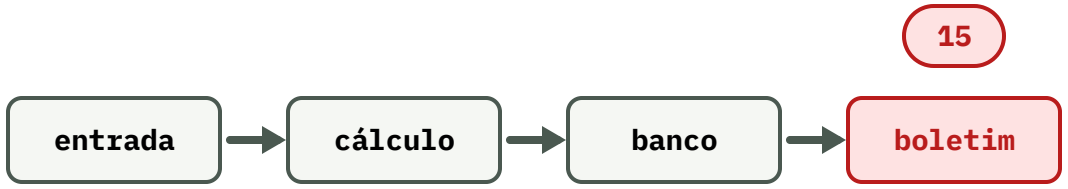
> Tipo novo = classe nova. O preco_final não muda: nada que já funcionava foi tocado.

Ponto 3: obsessão por tipos primitivos

Primitive Obsession · Wrap all primitives and strings

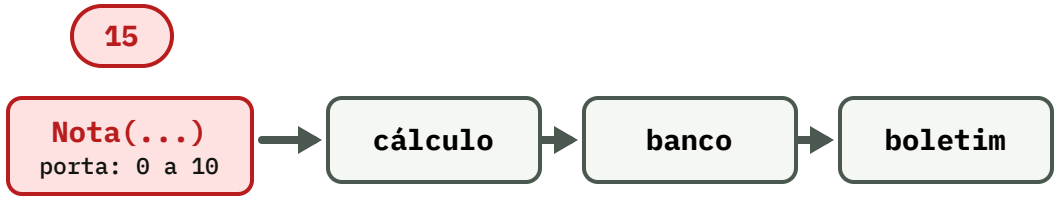
JEITO RUIM src/11/03_embrulhar_primiti… erro aparece: no fim

```
1 def registrar(ra, nota):
2     salvar(ra, nota)
3
4 registrar(8.5, 1234)      # trocou RA e nota
5 registrar(1234, 15)       # nota 15?
```



JEITO BOM src/11/03_embrulhar_prim… erro aparece: na entrada

```
1 class Nota:
2     def __init__(self, valor):
3         if valor < 0 or valor > 10:
4             raise ValueError("0 a 10")
5         self._valor = valor
6
7 registrar(Ra(1234), Nota(8.5))
8 Nota(15)      # recusa na entrada
```



> Mesmo valor 15: o jeito ruim leva até o fim; o bom barra na porta.

Ponto 4: coleção de primeira classe

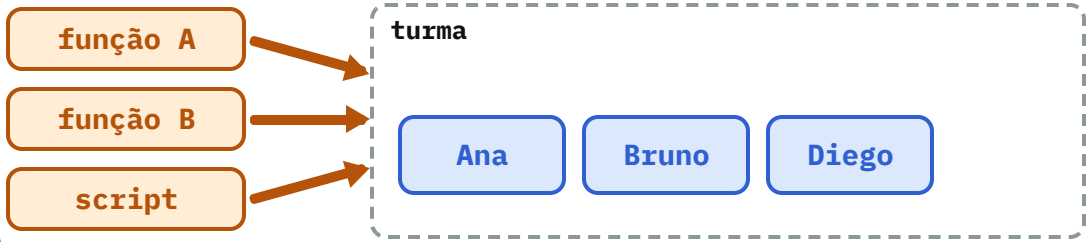
First-class collections

JEITO RUIM src/11/04_colecao_primeira_clas… **qualquer um mexe**

```
turma = []                          # lista solta
turma.append(Aluno("Ana", 8.5))

def aprovados(turma): ...           # qualquer função alcança

turma.append(None)                  # lixo entra
turma.clear()                       # e alguém apaga tudo
```

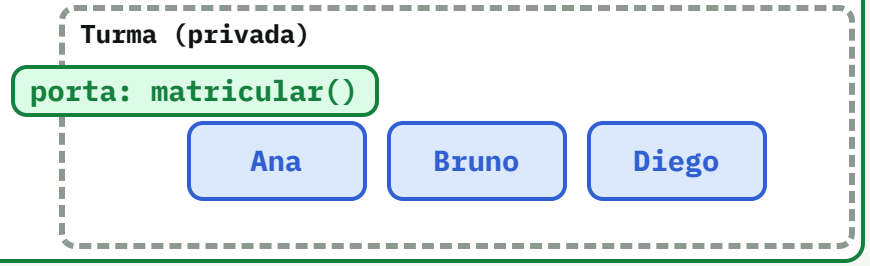


JEITO BOM src/11/04_colecao_primeira_classe.… **lista escondida**

```
class Turma:
    def __init__(self, alunos=()):
        self._alunos = list(alunos)

    def matricular(self, aluno):
        self._alunos.append(aluno)

    def aprovados(self):
        return Turma([aluno for aluno in self._alunos
                      if aluno.aprovado()])
```



> Mesma coleção: solta, qualquer um mexe; embrulhada, só as operações da turma.

Ponto 5: Lei de Demeter, um ponto por linha

Law of Demeter · One dot per line

JEITO RUIM src/11/05_um_ponto_por_linha.py **pontos: 4**

```python
# quem chama conhece a árvore inteira
disco = tf.pai.pai.pai.pai.nome

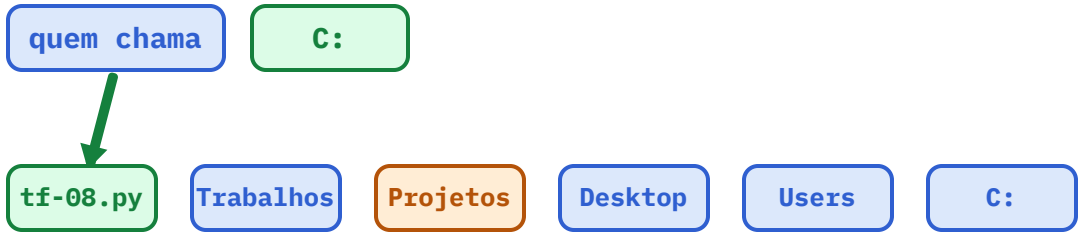
# a árvore ganhou um nível no meio...
# ...e o mesmo código devolve "Users"
```



JEITO BOM src/11/05_um_ponto_por_linha.py **pontos: 1**

```python
class Item:
    def disco(self):
        if self._pai is None:
            return self._nome
        return self._pai.disco()

disco = tf.disco()      # um ponto
```



> Mesma pergunta: 4 pontos que quebram contra 1 ponto que sobrevive à mudança.

Ponto 6: diga, não pergunte (sem getters e setters)

Tell, Don't Ask · No getters/setters/properties

JEITO RUIM src/11/06_diga_nao_pergunte.py qualquer um quebra

```python
if conta.get_saldo() >= 30:
    conta.set_saldo(conta.get_saldo() - 30)

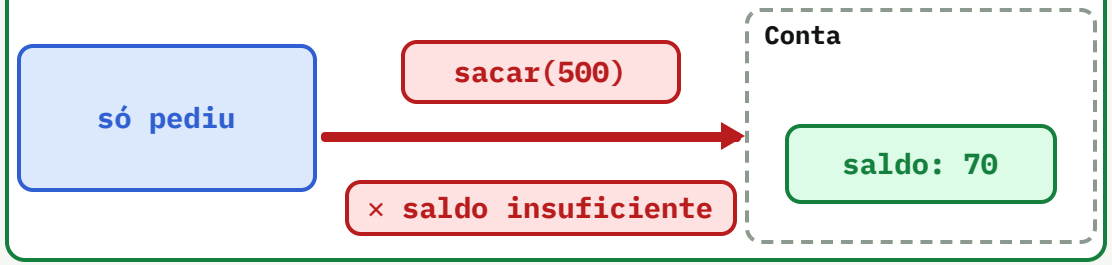
conta.set_saldo(-999)    # ninguém impede
```



JEITO BOM src/11/06_diga_nao_pergunte.… diz e a conta decide

```python
class Conta:
    def sacar(self, valor):
        if valor > self._saldo:
            raise ValueError("saldo insuficiente")
        self._saldo -= valor

conta.sacar(30)
conta.sacar(500)    # a conta recusa
```



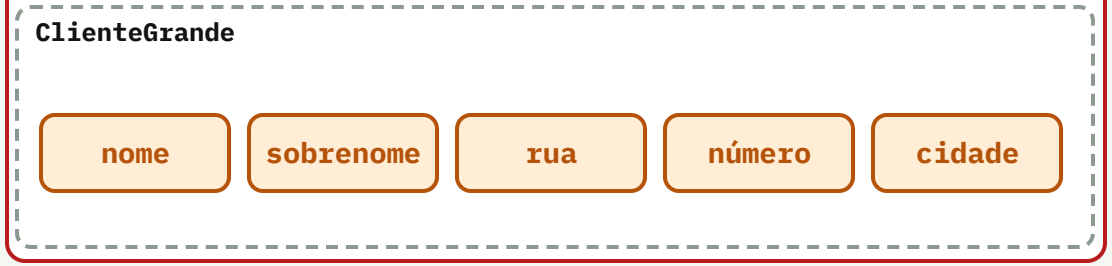
> Mesma conta: no jeito ruim a regra se espalha; no bom ela tem dono.

Ponto 7: entidades pequenas, até 2 variáveis

Keep all entities small · No classes with more than two instance variables

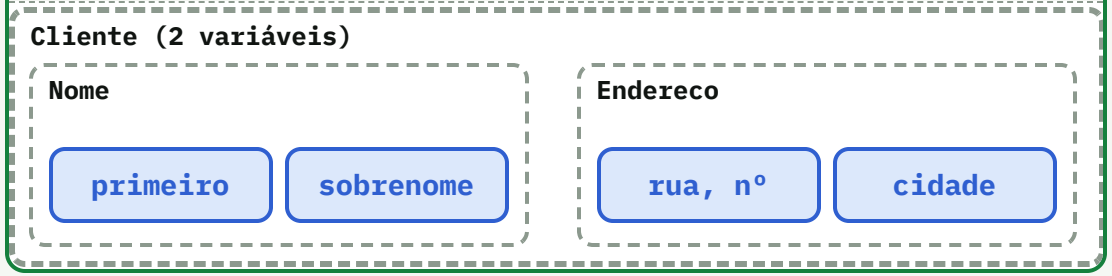
JEITO RUIM src/11/07_entidades_pequenas.py variáveis: 5

```
1 class ClienteGrande:
2     def __init__(self, nome, sobrenome,
3                  rua, numero, cidade):
4         self._nome = nome
5         self._sobrenome = sobrenome
6         self._rua = rua
7         self._numero = numero
8         self._cidade = cidade
```



JEITO BOM src/11/07_entidades_pequenas.py cada classe: 2

```
1 class Cliente:
2     def __init__(self, nome, endereco):
3         self._nome = nome
4         self._endereco = endereco
```



> Muitas classes pequenas, cada uma com um assunto, no lugar de uma gigante.

Resenha: regra de código só vale em equipe

- **Acordo escrito:** padrão de codificação com o **porquê** de cada regra. O XP (Kent Beck, 1999) já trata disso: padrão de codificação, propriedade coletiva do código e refatoração contínua
- **Revisão no pull request** (Aula 05): "isto fere o nosso acordo, e por quê", em vez de "eu faria diferente". O feedback vira critério compartilhado, não opinião
- **Automatizar o que dá:** no CI, `flake8 --max-complexity` e a regra `C901` do `ruff` pegam a escada de níveis. "Sem getters" e "um assunto só" ainda pedem julgamento humano
- **Praticar junto:** dojo, kata e programação em par. O aprendizado está na conversa: "por que você extraiu isso?"

Código que só o autor entende é uma **dependência de pessoa**: se ela sai, o conhecimento sai junto.

Dívida técnica: o empréstimo consciente

Technical debt · pegar velocidade agora, pagar depois com juros

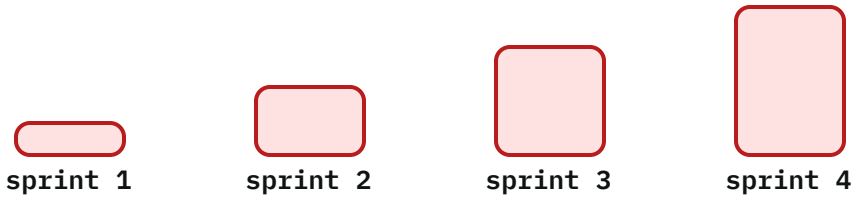
JEITO RUIM src/11/08_divida_tecnica.py — juros: só crescem

```python
# sexta, 17h: o cliente precisa hoje
def preco_aluno(preco):
    return preco * 0.5

def preco_professor(preco):
    return preco * 0.7

# copiei e colei: "depois eu arrumo"
```

custo de mexer aqui (os juros):

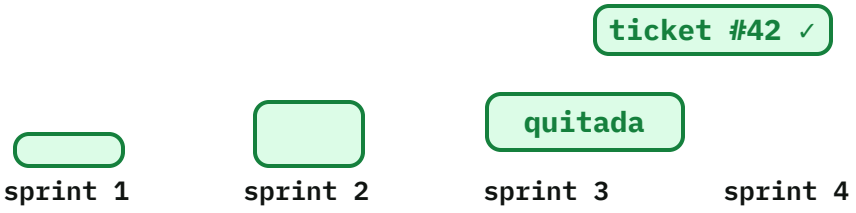


JEITO BOM src/11/08_divida_tecnica.py — quitada na sprint 3

```python
# Pagamento (sprint 3): nada duplicado.
MULTIPLICADOR = {"aluno": 0.5, "professor": 0.7}

def preco_final(preco, tipo):
    return preco * MULTIPLICADOR[tipo]
```

custo de mexer aqui (os juros):



> Mesma pressa, dois destinos: sem saber, os juros só crescem; sabendo, a dívida é registrada e paga.

Resenha: `processo` e custo da mudança

- **Cascata:** o desenho vem inteiro antes do código, e mudar custa cada vez mais com o tempo
- **Ágil** (XP, Scrum, Kanban): ciclos curtos, e o código precisa **continuar fácil de mudar** a cada ciclo. É aqui que as regras entram: mantêm o custo da mudança baixo
- **Dívida técnica:** cada atalho é um empréstimo, e a regra é o hábito de pagar à vista. Sem testes e sem refatoração contínua, o ágil vira só pressa
- Nenhuma regra substitui **conversar com quem vai usar**: código bonito resolvendo o problema errado continua sendo o problema errado

> **Para pensar:** qual regra mais incomodou, e o que ela escondia no seu código? O que aconteceria numa equipe de cinco **sem** acordo de código? E com um acordo que ninguém revisa?

Modularização: o guarda-chuva de todas as regras

"por que não usar else?" Porque falta separar melhor a unidade de execução

JEITO RUIM src/11/09_modularizacao.py **unidades: 1**

```
1 def relatorio_antes(turma):
2     for nome in turma:
3         if len(turma[nome]) > 0:
4             soma = 0
5             for nota in turma[nome]:
6                 soma += nota
7             media = soma / len(turma[nome])
8             if media >= 6:
9                 print(nome, "aprovado")
```

pilha de execução (stack):

relatorio_antes()
percorre · soma · média · decide · imprime

erro
em qual das 5?

JEITO BOM src/11/09_modularizacao.py **unidades: 3**

```
1  def media(notas):
2      return sum(notas) / len(notas)
3
4  def foi_aprovado(notas):
5      return len(notas) > 0 and media(notas) >= 6
6
7  def relatorio(turma):
8      for nome in turma:
9          if foi_aprovado(turma[nome]):
10             print(nome, "aprovado")
```

pilha de execução (stack):

media()
só a conta

foi_aprovado()
decide

relatorio()
percorre e imprime

> Mesmo programa. Modularizar é separar as unidades de execução: cada uma faz uma coisa e a pilha vira uma história legível.

tarefa

TF 11: a loja feia

Num repositório novo `tf-11`, com venv e `.gitignore`, pegar o código feio `src/11/tf-11/loja_feia.py` do repositório base e **consertá-lo em passos**, sem mudar o que ele imprime:

- **Rede de segurança:** guardar a saída original e rodar um `diff` depois de cada passo
- **Passos:** nomes sem abreviar, **normalização** (Cliente, Produto), **obsessão por tipos primitivos** (Cpf, Email, Cep, Dinheiro, Status), coleções (ItensDoPedido, Pedidos), entidades pequenas, **Lei de Demeter** e diga-não-pergunte, sem `else` (**fail first** e polimorfismo)
- **README:** normalização, primitivos, modularização (uma linha por classe) e a **dívida técnica** que sobrou
- Um commit por passo (8 ou mais), repositório **Public**, sem `.venv`, trabalho individual

Entrega: link do repositório no GitHub (público), pelo formulário abaixo — `https://docs.google.com/forms/d/e/1FAIpQLSf-SrLqtjyG_L1fQTq2o1ULjmE0HTPzSD7BX_4vSCvvqUe0Yg/viewform?usp=publish-editor`

Prazo: **11/11, até 23:59**.